

Topic Five: Fault Tolerance in Distributed Systems

Topic Outline

- Types of failures: crash, omission, Byzantine
- Redundancy and replication
- Check-pointing and recovery
- Consensus algorithms: Paxos, Raft

Introduction

Fault Tolerance in Distributed Systems

Fault tolerance is the property of a system that allows it to continue functioning correctly even when some components fail. In distributed systems, failures are expected rather than exceptional. Fault tolerance is needed to ensure that a system continues to operate correctly even when some of its components fail. In today's world—where businesses, healthcare, banking, transportation, and cloud services depend heavily on technology—failures are inevitable. Fault tolerance minimizes disruption and protects data, money, and lives.

Here are the key reasons why fault tolerance is important:

1. Ensures High Availability

Systems can remain operational even if hardware, software, or network components fail. For example, cloud platforms like Amazon Web Services use redundancy so services stay online even if a server crashes.

2. Prevents Data Loss

Fault-tolerant systems replicate and back up data across multiple locations. Companies like Google duplicate data across data centers to protect against hardware failures.

3. Improves System Reliability

It increases user trust by reducing downtime and errors. In industries like banking or healthcare, even a few minutes of downtime can cause major losses.

4. Supports Critical Applications

Some systems must never fail, such as:

- Air traffic control systems
- Medical life-support systems
- Financial transaction processing systems

Fault tolerance ensures these continue functioning even when parts fail.

5. Handles Hardware & Software Failures

Failures can occur due to:

- Power outages
- Disk crashes
- Network failures
- Software bugs

Fault tolerance detects faults and switches to backup components automatically.

6. Reduces Financial Loss

Downtime can cost companies thousands or millions of dollars per hour. Fault tolerance helps avoid revenue loss and reputational damage.

7. Maintains Business Continuity

It ensures organizations can continue operations during unexpected events like cyberattacks or natural disasters.

1. Types of Failures

1. Crash Failures

A crash failure occurs when a node stops functioning and does not respond anymore.

Characteristics:

- Node halts and remains silent.
- Does not send incorrect messages.
- Easier to detect (timeouts, heartbeats).

Example:

- A database server in a cloud cluster loses power.
- A VM instance is terminated unexpectedly.

Real-World Example:

- In a cluster running consensus via Raft, if the leader crashes:
 - Followers detect missing heartbeats.
 - A new leader is elected.
 - System continues operating.

Handling Techniques:

- Heartbeat messages
- Leader election
- Replication
- Automatic restart

2. Omission Failures

Omission failures happen when messages are not sent or received.

Types:

1. Send omission – Node fails to send a message.
2. Receive omission – Node fails to receive a message.
3. Channel omission – Network drops messages.

Causes:

- Packet loss
- Network congestion
- Faulty network interfaces
- Buffer overflow

Example:

- A microservice sends a request but never receives a response due to network drop.
- A payment request times out even though the service is running.

Handling:

- Retransmissions
- Acknowledgements (ACKs)
- Timeout mechanisms
- Idempotent operations (safe retries)

Real-World Example:

TCP protocol handles omission failures using:

- Sequence numbers
- ACKs
- Retransmission on timeout

3. Byzantine Failures

Byzantine failures are arbitrary or malicious faults.

A node may:

- Send conflicting data to different nodes
- Modify data incorrectly
- Act unpredictably
- Be compromised by attackers

This problem originates from the Byzantine Generals Problem.

Difficulty:

Hardest failure type to tolerate.

The $3f + 1$ Rule:

To tolerate f Byzantine nodes, the system needs at least $3f + 1$ total nodes.

Example:

If 1 node may be malicious:

- At least 4 nodes are required.

Real-World Example:

Blockchain systems must tolerate Byzantine behavior because nodes:

- May be malicious
- May try to double-spend
- May send conflicting transactions

2. Redundancy and Replication

Fault tolerance relies on redundancy.

Redundancy = Extra components added so failure does not stop the system.

Types of Redundancy

1. Hardware Redundancy

- Multiple servers
- RAID storage
- Backup power supplies

Example:

Data centers use multiple power feeds to prevent outage.

2. Information Redundancy

- Checksums
- Parity bits
- Error-correcting codes (ECC memory)

Example:

ECC memory detects and corrects single-bit memory errors.

3. Time Redundancy

- Retry failed operations
- Re-execute tasks

Example:

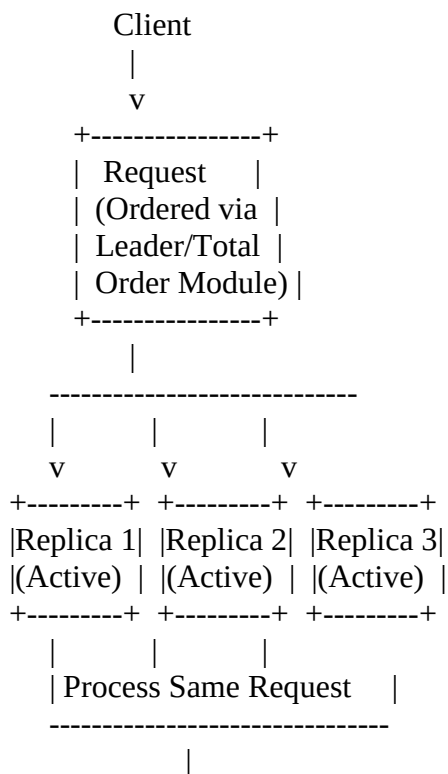
Cloud storage retries upload when packet loss occurs.

Replication

Replication means maintaining copies of data/services across multiple nodes.

1. Active Replication (State Machine Replication)

- All replicas process the same requests in the same order.
- Used in consensus-based systems.
- Ensures strong consistency.



v
Client gets reply
(usually from one replica)

Steps in Active Replication

Step 1: Client Sends Request

The client sends a request to the system (often to a leader or front-end).

Step 2: Request Ordering

A leader or ordering service ensures **total order of requests**.
All replicas must process requests in exactly the same order.

Step 3: Request Broadcast

The ordered request is broadcast to all replicas.

Step 4: Parallel Execution

All replicas:

- Execute the same request
- Perform the same computation
- Update their internal state identically

Step 5: Response to Client

- One replica (often the leader) sends the response to the client.
- Other replicas discard or suppress their replies.

Step 6: Fault Handling

If one replica crashes:

- Other replicas continue working.
- Since all replicas maintain identical state, service continues without interruption.

All replicas:

- Execute the same request
- Perform the same computation

Key Requirements

1. Deterministic Operations

All replicas must produce the same output for the same input.

2. Same Order of Execution

- Ensured by consensus or ordering protocols.
- Update their internal state identically

Example:

In systems using Paxos:

- Every node executes commands in identical order.
- If one node crashes, others continue.

Advantages:

- High availability
- Strong consistency

Disadvantages:

- Higher overhead
- Requires deterministic operations

2. Passive Replication (Primary-Backup)

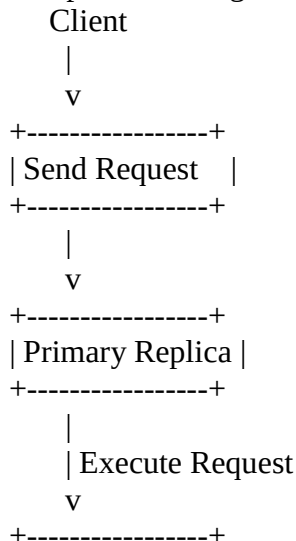
- One primary handles client requests.
- Backups replicate state.
- On failure, backup becomes primary.

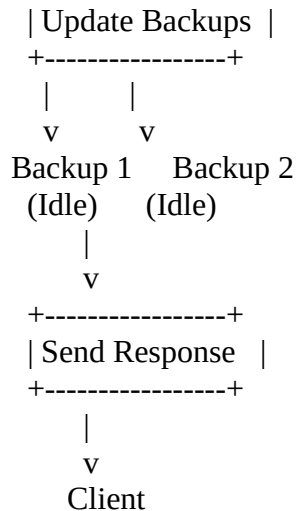
Example:

Traditional relational database clusters.

Flow:

1. Client sends request to primary.
2. Primary updates state.
3. Primary sends state to backup.
4. Backup acknowledges.





Steps in Passive Replication

Step 1: Client Sends Request

- The client sends a request **only to the primary replica**.

Step 2: Primary Executes Request

- The primary replica performs the computation and updates its internal state.

Step 3: Primary Updates Backups

- The primary sends the updated state (or logs of operations) to backup replicas.
- Backups apply the updates **without executing the request themselves**.

Step 4: Primary Sends Response

- The primary sends the result back to the client.
- Backups do not respond to the client.

Step 5: Failover (if needed)

- If the primary fails, one backup is promoted to primary.
- New primary continues processing requests using the latest consistent state from the previous primary.

Key Points

- Only **primary executes requests**, backups are passive.
- **Lower resource usage** compared to active replication.
- **Failover may introduce some delay** while a backup takes over.
- **Consistency** is maintained because backups get state updates from the primary.

3. Checkpointing and Recovery

Checkpointing = Periodically saving the system state to stable storage.

Why Checkpointing is needed

If a system crashes:

- Without checkpointing → restart from beginning.
- With checkpointing → restore from last saved state.

Types of Checkpointing

1. Uncoordinated Checkpointing

- Each process saves state independently.
- May create inconsistent global states.

Problem:

Domino effect – cascading rollbacks.

Example:

Process A rolls back → forces B → forces C → system returns to very old state.

2. Coordinated Checkpointing

- Processes coordinate to create consistent global snapshot.
- Avoids domino effect.

Example:

Distributed database periodically pauses writes and takes snapshot.

3. Incremental Checkpointing

- Only changes since last checkpoint are stored.
- Reduces storage overhead.

Example:

Cloud VM snapshot storing only delta changes.

Log-Based Recovery

Instead of only saving state:

- Log every operation.
- On recovery:
 - Restore checkpoint
 - Replay log

Example:

Banking system:

1. Checkpoint at 10:00 AM
2. Crash at 10:10 AM

3. Replay transactions from 10:00 to 10:10

4. Consensus Algorithms

Consensus ensures nodes agree on a single value despite failures.

Requirements:

- Agreement
- Validity
- Termination

Paxos

Developed by Leslie Lamport.

Goal:

Reach agreement in presence of crash failures.

Roles:

- Proposer
- Acceptor
- Learner

Phases:

1. Prepare phase
2. Accept phase

Guarantees:

- Safety always guaranteed.
- Liveness under certain conditions.

Real-World Use:

- Distributed databases
- Configuration management systems

Limitation:

- Hard to understand and implement.

Raft

Designed to be easier than Paxos.

Key Concepts:

- Strong leader
- Log replication
- Leader election
- Terms (logical time periods)

Working:

1. Leader receives client request.
2. Leader appends to its log.
3. Sends entry to followers.
4. Majority acknowledgment → commit.

Example Scenario:

If leader crashes:

- Followers detect missing heartbeat.
- New election triggered.
- New leader continues log.

Advantages:

- Easier to implement.

- Clear separation of election and replication.

Used in:

- Distributed key-value stores
- Cluster management systems

5. Fault Tolerance vs Availability vs Reliability

Concept	Meaning
Reliability	Probability system works correctly over time
Availability	System is operational when needed
Fault Tolerance	System continues operating despite failures

Example:

A system may be reliable but not fault tolerant (crashes completely on failure).

6. Practical Example: Distributed Database Failure Scenario

Consider a 5-node cluster using Raft:

If:

- 1 node crashes → system continues.
- 2 nodes crash → still works (majority = 3).
- 3 nodes crash → system stops (no majority).

This demonstrates majority-based fault tolerance.